Index of the First Occurrence of pattern in a text

Bruteforce

```python
class Solution:
    def findMatching(self, text, pattern):
        n= len(text)
        m= len(pattern)

        for i in range(n-m +1):
            match= True
            for j in range(m):
                if text[i+j] != pattern[j]:
                    match=False
                    break
            if match:
                return i
        return -1
```

Naive Solution

text = "gbbgbg"

pattern = "gbg"

index :	0	1	2	3	4	5
Text :	g	b	b	g	b	g
Pattern						

length = 3

$n = \text{len}(\text{text})$; $n = 6$
$m = \text{len}(\text{pattern})$; $m = 3$

Loop : i from $0 \rightarrow n - m$

$i = 0 \rightarrow 3$

Step 1 ($i = 0$)

Compare substring starting at 0

text[0] = g pattern[0] = g ✓
text[1] = f pattern[1] = f ✓
text[2] = f pattern[2] = g ✗

Mismatch → move to next index

Step 2 ($i = 1$)
Substring starting at 1

text[1] = f pattern[0] = g ✗

Mismatch immediately

Step 3 ($i = 2$)
Substring starting at 2
text[2] = f pattern[0] = g ✗

Mismatch

Step 4 ($i = 3$)

Substring starting at 3

text[3] = g pattern[0] = g

text[4] = f pattern[1] = f

text[5] = g pattern[2] = g

ALL Characters match

Return 3

Bruteforce Iterations Summary

i	Substring	Result
0	gff	X
1	ffg	X
2	fgf	X
3	gfg	

→ Answer = 3

TC → $O(n * m)$

SC → $O(1)$

Optimal Solution (KMP Algorithm)

```python
class Solution:
    def findMatching(self, text, pattern):
        n= len(text)
        m= len(pattern)
        lps=[0] * m
        length =0
        i=1

        while i<m:
            if pattern[i] == pattern[length]:
                length +=1
                lps[i]=length
                i+=1
            else:
                if length !=0:
                    length = lps[length-1]
                else:
                    lps[i]=0
                    i+=1

        i=j=0
        while i<n:
            if text[i] == pattern[j]:
                i+=1
                j+=1
            if j==m:
                return i-j

            elif i<n and text[i] != pattern[j]:
                if j!=0:
                    j= lps[j-1]
                else:
                    i+=1
        return -1
```

KMP has two steps

1. Build LPS Array
2. Search using LPS

Step 1 ⇒ Build LPS Array

Pattern

$\underset{0}{g}, \underset{1}{f}\ \underset{2}{g}$

LPS[i] = Longest Prefix that is also Suffix

Initialize

lps = [0, 0, 0]

i = 1

Compare

pattern[1] = f

pattern[0] = g

Not equal

lps[1] = 0

$i = 2$

pattern[2] = g
pattern[0] = g

match

length = 1
lps[2] = 1

Final LPS

Pattern =	g	f	g
Index	0	1	2
LPS	0	0	1

Step2 → Search in Text

Pointers
i → next
j → pattern

Start
$i = 0$
$j = 0$

Step 1 text[0] = g

pattern[0] = g

match $i = 1, j = 1$

Step 2

text[1] = b

pattern[1] = b

Match $i = 2, j = 2$

Step - 3

text[2] = b

pattern[2] = g

Mismatch

Using LPS

$j = lps[j-1]$

$j = lps[1] = 0$

Now compare again

Step-4

text[2] = f

pattern[0] = g

mismatch

i++

i = 3

Step 5

text[3] = g

pattern[0] = g

Match

i = 4, j = 1

Step-6

text[4] = f

pattern[1] = f

Match i = 5, j = 2

Step-7

text[5] = g

pattern[2] = g

Match $i = 6, j = 3$

Now,

j == pattern length

Return $i - j = 6 - 3 = 3$

TC → $O(n * m)$

SC → $O(1)$